Introduction à la programmation événementielle - Événements liés à la souris

Nous avons vu l'utilisation du addEventListener de manière générale. Nous allons maintenant nous concentrer sur les événements liés à la souris.

Détecter un clic

Le plus simple pour détecter un clic sur le canvas ou un élément HTML est d'utiliser l'événement `click`.

```
const canvas = document.getElementById("monCanvas");
canvas.addEventListener("click", clicSurCanvas);

function clicSurCanvas(evenement) {
    let curseurX = evenement.clientX;
    let curseurY = evenement.clientY;

    console.log(`Clic détecté à la position (${curseurX}, ${curseurY}) par rapport
à la fenêtre du navigateur.`);
}
```

L'objet événement

Lorsque l'événement est déclenché, une fonction de rappel (callback) est toujours appelée avec un objet événement en paramètre. Cet objet contient des informations sur l'événement, comme la position du curseur, le bouton de la souris utilisé, etc.

Voici quelques propriétés couramment utilisées de l'objet événement pour les événements liés à la souris :

- `clientX` et `clientY` : La position du curseur par rapport à la fenêtre du navigateur.
- `offsetX` et `offsetY` : La position du curseur par rapport à l'élément cible de l'événement.
- `button` : Le bouton de la souris qui a été cliqué (0 pour le bouton gauche, 1 pour le bouton du milieu, 2 pour le bouton droit).
- `altKey`, `ctrlKey`, `shiftKey` : Indiquent si les touches Alt, Ctrl ou Shift étaient enfoncées lors de l'événement.
- `currentTarget` : L'élément HTML sur lequel l'événement a été attaché.
- `type` : Le type d'événement (par exemple, "click", "mousemove", etc.).

```
function clicSurCanvas(evenement) {
    let curseurPageX = evenement.clientX;
    let curseurPageY = evenement.clientY;
    let curseurCanvasX = evenement.offsetX;
    let curseurCanvasY = evenement.offsetY;
    let bouton = evenement.button;
    let altEnfoncee = evenement.altKey;
    let ctrlEnfoncee = evenement.ctrlKey;
    let shiftEnfoncee = evenement.shiftKey;
```

```
    let elementCible = evenement.currentTarget;
    let typeEvenement = evenement.type;

    console.log(`Clic détecté à la position (${curseurX}, ${curseurY}) par rapport
à la fenêtre du navigateur.`);
    console.log(`Clic détecté à la position (${curseurOffsetX}, ${curseurOffsetY})
par rapport au bord du canvas.`);
    console.log(`Bouton cliqué : ${bouton}`); // 0 : gauche, 1 : milieu, 2 :
droite
    console.log(`Alt enfoncée : ${altEnfoncee}, Ctrl enfoncée : ${ctrlEnfoncee},
Shift enfoncée : ${shiftEnfoncee}`);
    console.log(`Élément cible :`, elementCible);
    console.log(`Type d'événement : ${typeEvenement}`);
}
```

Récupérer la position du curseur au moment du clic par rapport au canvas

Pour récupérer la position du curseur par rapport au canvas, on récpère dans un premier temps la position du curseur par rapport à la fenêtre du navigateur, puis on soustrait la position du canvas par rapport à la fenêtre du navigateur. Cela nous donne la position du curseur par rapport au canvas.

```
const oCanvasHTML = document.getElementById("monCanvas");

function initialiser() {
    oCanvasHTML.addEventListener("click", clicSurCanvas);
}

function clicSurCanvas(evenement) {
    let positionCurseurX = evenement.offsetX;
    let positionCurseurY = evenement.offsetY;

    console.log(`Clic détecté à la position (${positionCurseurX},
${positionCurseurY}) par rapport au canvas.`);
}
```

Détecter la collision entre un objet et le curseur de la souris

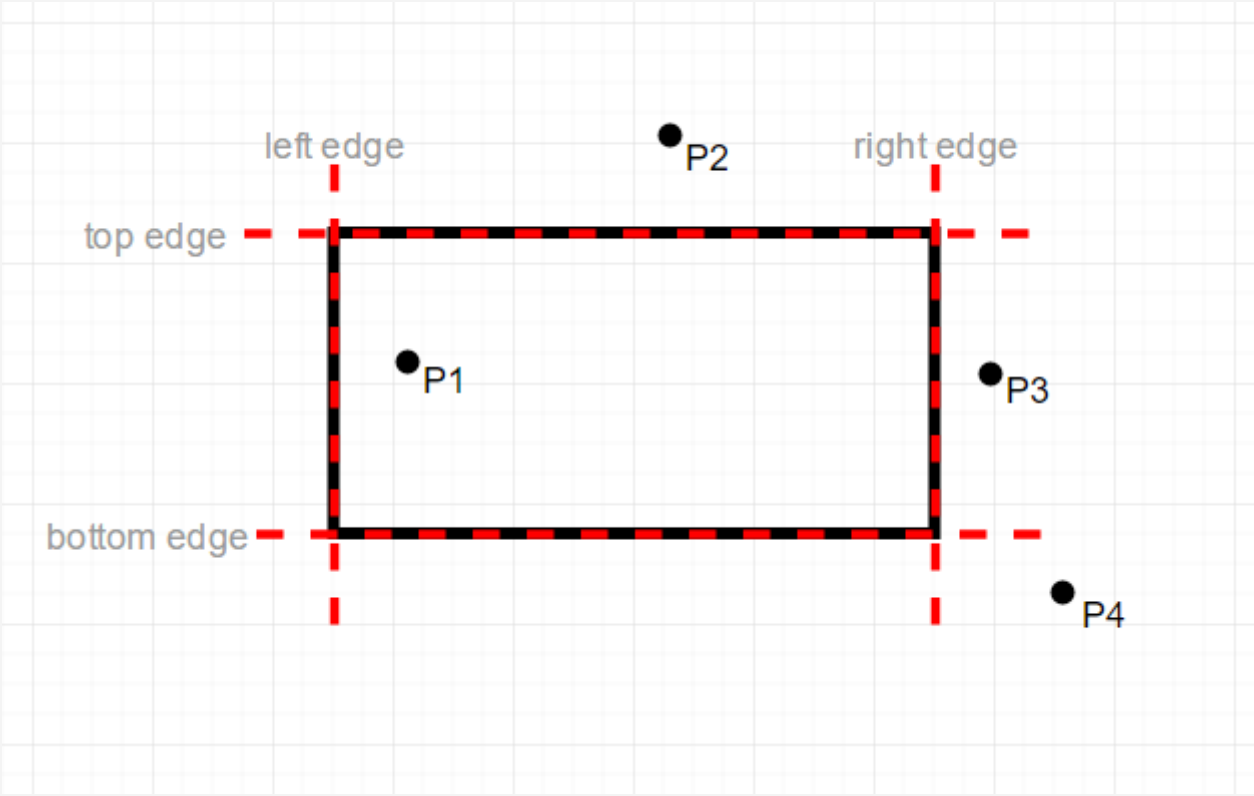
La détection de collision entre le curseur de la souris et un objet est une tâche courante dans les applications interactives. Voici un exemple simple de détection de collision entre le curseur et un rectangle dessiné sur le canvas.

Comme avec le canvas, tout est dessiné, il est impossible de savoir sur quel élément du jeu on a cliqué. Il faut donc faire la détection de collision "à la main".

Le concept est simple : on vérifie si la position du curseur se trouve à l'intérieur des coordonnées de l'objet. C'est à dire que le curseur doit être:

1. à droite du bord gauche de l'objet
2. à gauche du bord droit

3. en dessous du bord supérieur
4. au-dessus du bord inférieur.




```
    SI positionClicX est supérieur à son côté gauche(positionObjetX) ET
    positionClicX est inférieur à son côté droit(positionObjetX + largeurObjet) ET
    positionClicY est supérieur à son bord haut(positionObjetY) ET
    positionClicY est inférieur à son bord bas(positionObjetY + hauteurObjet)
        ALORS il y a collision
    FIN SI
```

```
function detecterCollision(positionClicX, positionClicY, positionObjetX,
positionObjetY, largeurObjet, hauteurObjet) {
    let estEnCollision;
    if (
        positionClicX >= positionObjetX &&
        positionClicX <= positionObjetX + largeurObjet &&
        positionClicY >= positionObjetY &&
        positionClicY <= positionObjetY + hauteurObjet
    ) {
        estEnCollision = true;
    } else {
        estEnCollision = false;
    }
}
```

Suivre la position du curseur en temps réel

L'événement `mousemove` permet de suivre la position du curseur en temps réel lorsqu'il se déplace sur un élément HTML, comme un canvas.

Dans l'exemple suivant, nous allons dessiner une image (un ballon) qui est collée au curseur de la souris lorsqu'il se déplace sur le canvas.

```
const oCanvasHTML = document.querySelector("canvas");
let oContexte = oCanvasHTML.getContext("2d");

let oImageBallon = new Image();
oImageBallon.src = "images/ballon.png";

let positionXBallon = 100;
let positionYBallon = 100;
let largeurBallon = 50;
let hauteurBallon = 50;

function initialiser() {
    oCanvasHTML.addEventListener("mousemove", deplacerCurseur);
    setInterval(boucleJeu, 1000 / 60); // Boucle de jeu à 60 FPS
}

function boucleJeu() {
    oContexte.clearRect(0, 0, oCanvasHTML.width, oCanvasHTML.height);
    oContexte.drawImage(oImageBallon, positionXBallon, positionYBallon);
}

function deplacerCurseur(evenement) {
    let curseurCanvasX = evenement.offsetX;
    let curseurCanvasY = evenement.offsetY;

    positionXBallon = curseurCanvasX - largeurBallon / 2;
    positionYBallon = curseurCanvasY - hauteurBallon / 2;
}
```

Changer le curseur de la souris lorsqu'il est au-dessus du canvas

Vous pouvez utiliser la propriété CSS `cursor` pour changer l'apparence du curseur lorsque la souris est au-dessus du canvas.

Si vous mettez `cursor: none`, le curseur disparaît lorsque la souris est au-dessus du canvas.

```
canvas {
    border: 1px solid black;
    cursor: none; /* Le curseur disparaît au-dessus du canvas */
}
```

Si vous voulez utiliser une image personnalisée comme curseur, vous pouvez définir la propriété `cursor` avec l'URL de l'image. Cependant, l'image doit être au format `.png`, `.cur` ou `.gif` et avoir une taille

maximale recommandée de 32x32 pixels pour de meilleurs résultats. On indique aussi une valeur de repli (comme `auto` ou `pointer`) au cas où l'image ne pourrait pas être chargée.

```
canvas {
    border: 1px solid black;
    cursor: url("../images/mon_curseur_personnalise.png"), crosshair; /* Utilise
une image personnalisée comme curseur */
}
```

Voici la liste des valeurs possibles pour la propriété `cursor` :

- `auto` : Le navigateur choisit le curseur par défaut.
- `default` : Le curseur par défaut (généralement une flèche).
- `pointer` : Un curseur en forme de main, souvent utilisé pour les liens.
- `crosshair` : Un curseur en forme de croix.
- `move` : Un curseur indiquant que l'élément peut être déplacé.
- `text` : Un curseur en forme de I-beam, utilisé pour indiquer du texte sélectionnable.
- `wait` : Un curseur en forme de sablier ou de roue qui tourne, indiquant que l'utilisateur doit attendre.
- `not-allowed` : Un curseur indiquant que l'action n'est pas autorisée (généralement un cercle barré).
- `none` : Le curseur disparaît.